

EXP NO:1a

DATE:

Prime Number Validation for RSA Key Generation

Aim:

Algorithm:

231401028

Program:

RSA Prime Number Validation

```
def isPrime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

p = int(input("Enter the first number (p): "))
q = int(input("Enter the second number (q): "))

if isPrime(p):
    print(p, "is a Prime Number.")
else:
    print(p, "is NOT a Prime Number.")

if isPrime(q):
    print(q, "is a Prime Number.")
else:
    print(q, "is NOT a Prime Number.")

if isPrime(p) and isPrime(q):
    print("\nBoth numbers are prime.")
    print("Proceed with RSA Key Generation.")
else:
    print("\nInvalid Prime Number(s).")
    print("RSA Key Generation Aborted.")
```

Output:

```
Enter the first number (p): 18
Enter the second number (q): 29
18 is NOT a Prime Number.
29 is a Prime Number.

Invalid Prime Number(s).
RSA Key Generation Aborted.
```

Result:

EXP NO:1b	Secure Communication Key Validation
DATE:	

Aim:

Algorithm:

231401028

Program:

```
def find_gcd(a, b):
    if a < b:
        a, b = b, a
    print("\nSteps of Euclidean Algorithm:")
    while b != 0:
        remainder = a % b
        print(f"{a} ÷ {b} = Quotient {a//b}, Remainder {remainder}")
        a = b
        b = remainder
    return a

print("=== Secure Communication Key Validation ===")
parameter1 = int(input("Enter the first encryption parameter: "))
parameter2 = int(input("Enter the second encryption parameter: "))

gcd = find_gcd(parameter1, parameter2)
print("\nGreatest Common Divisor (GCD) =", gcd)

if gcd == 1:
    print("\nThe encryption parameters are COPRIME.")
    print("Secure communication can be established.")
else:
    print("\nThe encryption parameters are NOT COPRIME.")
    print("Communication setup rejected.")
    print("Choose different encryption parameters.")
```

Output:

```
=== Secure Communication Key Validation ===  
Enter the first encryption parameter: 35  
Enter the second encryption parameter: 12
```

Steps of Euclidean Algorithm:

$35 \div 12 = \text{Quotient } 2, \text{ Remainder } 11$

$12 \div 11 = \text{Quotient } 1, \text{ Remainder } 1$

$11 \div 1 = \text{Quotient } 11, \text{ Remainder } 0$

Greatest Common Divisor (GCD) = 1

The encryption parameters are COPRIME.

Secure communication can be established.

Result:

EXP NO:1c

DATE:

Secure Message Encryption Using Modular Arithmetic

Aim:

Algorithm:

231401028

Program:

```
print("=====")
print(" MODULAR ARITHMETIC CALCULATOR")
print("=====")

a = int(input("Enter the First Integer (a): "))
b = int(input("Enter the Second Integer (b): "))
m = int(input("Enter the Modulus (m): "))

if m <= 0:
    print("Modulus must be greater than 0.")
else:
    while True:
        print("\n===== MENU =====")
        print("1. Modular Addition")
        print("2. Modular Subtraction")
        print("3. Modular Multiplication")
        print("4. Modular Exponentiation")
        print("5. Exit")
        choice = int(input("Enter your choice (1-5): "))

        if choice == 1:
            result = (a + b) % m
            print("\nModular Addition")
            print(f"({a} + {b}) mod {m} = {result}")
        elif choice == 2:
            result = (a - b) % m
            print("\nModular Subtraction")
            print(f"({a} - {b}) mod {m} = {result}")
        elif choice == 3:
            result = (a * b) % m
            print("\nModular Multiplication")
            print(f"({a} × {b}) mod {m} = {result}")
        elif choice == 4:
            result = pow(a, b, m)
            print("\nModular Exponentiation")
            print(f"({a}^{b}) mod {m} = {result}")
        elif choice == 5:
            print("\nExiting the Program...")
            break
        else:
            print("\nInvalid Choice! Please select between 1 and 5.")
```

Output:

```
=====
MODULAR ARITHMETIC CALCULATOR
=====
Enter the First Integer (a): 15
Enter the Second Integer (b): 7
Enter the Modulus (m): 11

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 1

Modular Addition
(15 + 7) mod 11 = 0
```

Result:

EXP NO:1d

DATE:

RSA Prime Number Validation Using Fermat's Little Theorem

Aim:

Algorithm:

231401028

Program:

```
print("=====")
print(" RSA Prime Validation using Fermat's Test ")
print("=====")

a = int(input("Enter the value of a: "))
p = int(input("Enter the candidate prime number (p): "))

if p <= 1:
    print("\nInvalid Input!")
    print("The candidate number p must be greater than 1.")
else:
    result = pow(a, p-1, p)
    print("\nResult of Fermat Test")
    print(f"({a}^{p-1}) mod {p} = {result}")

    if result == 1:
        print("\nThe number satisfies Fermat's Little Theorem.")
        print("It is a probable prime.")
        print("Candidate is suitable for further RSA key generation.")
    else:
        print("\nThe number does not satisfy Fermat's Little Theorem.")
        print("It is composite.")
        print("Candidate is NOT suitable for RSA key generation.")
```

Output:

```
=====
RSA Prime Validation using Fermat's Test
=====
Enter the value of a: 2
Enter the candidate prime number (p): 17

Result of Fermat Test
(2^16) mod 17 = 1

The number satisfies Fermat's Little Theorem.
It is a probable prime.
Candidate is suitable for further RSA key generation.
```

Result:

EXP NO:1e

DATE:

Digital Banking Security using Euler's Totient Function and Euler's Theorem

Aim:

Algorithm:

231401028

Program:

```
# Digital Banking Security using Euler's Totient Function and Euler's Theorem
import math
```

```
# Input values
```

```
a = int(input("Enter the value of a: "))
```

```
n = int(input("Enter the value of n: "))
```

```
# Step 1: Compute Euler's Totient Function  $\phi(n)$ 
```

```
phi = 0
```

```
for i in range(1, n + 1):
```

```
    if math.gcd(i, n) == 1:
```

```
        phi += 1
```

```
print("\nEuler's Totient Function")
```

```
print("φ(", n, ") =", phi)
```

```
# Step 2: Check whether a and n are coprime
```

```
if math.gcd(a, n) == 1:
```

```
    print(a, "and", n, "are coprime.")
```

```
# Step 3: Verify Euler's Theorem
```

```
result = pow(a, phi, n)
```

```
print("\nEuler's Theorem Verification")
```

```
print(f"{a}^{phi} mod {n} = {result}")
```

```
if result == 1:
```

```
    print("Euler's Theorem is Verified.")
```

```
else:
```

```
    print("Euler's Theorem is NOT Verified.")
```

```
else:
```

```
    print(a, "and", n, "are not coprime.")
```

```
    print("Euler's Theorem cannot be applied.")
```

Output:

```
Enter the value of a: 3
Enter the value of n: 10

Euler's Totient Function
 $\phi(10) = 4$ 
3 and 10 are coprime.

Euler's Theorem Verification
 $3^4 \bmod 10 = 1$ 
Euler's Theorem is Verified.
```

Result:

EXP NO:1 Extra	Military Secure Communication Using Coprime Verification
DATE:	

Aim:

Algorithm:

231401028

Program:

```
print("=====")
print(" Military Secure Communication System")
print("=====")

parameter1 = int(input("Enter the First Communication Parameter: "))
parameter2 = int(input("Enter the Second Communication Parameter: "))

a = parameter1
b = parameter2

while b != 0:
    remainder = a % b
    print(f"{a} ÷ {b} = Quotient {a//b}, Remainder {remainder}")
    a = b
    b = remainder

gcd = a
print("\nGreatest Common Divisor (GCD) =", gcd)

if gcd == 1:
    print("\nThe communication parameters are COPRIME.")
    print("Secure communication can be established.")
else:
    print("\nThe communication parameters are NOT COPRIME.")
    print("Communication setup rejected.")
    print("Select different communication parameters.")
```

Output:

```
=====
Military Secure Communication System
=====
Enter the First Communication Parameter: 25
Enter the Second Communication Parameter: 14
25 ÷ 14 = Quotient 1, Remainder 11
14 ÷ 11 = Quotient 1, Remainder 3
11 ÷ 3 = Quotient 3, Remainder 2
3 ÷ 2 = Quotient 1, Remainder 1
2 ÷ 1 = Quotient 2, Remainder 0

Greatest Common Divisor (GCD) = 1

The communication parameters are COPRIME.
Secure communication can be established.
```

Result:

EXP NO:2a

DATE:

Linear Congruential Generator (LCG)

Aim:

Algorithm:

231401028

Program:

```
# Linear Congruential Generator (LCG)
seed = 3
a = 5
c = 2
m = 13
n = 5

sequence = [seed]
x = seed

print("Linear Congruential Generator")
print("-----")
for i in range(1, n + 1):
    x = (a * x + c) % m
    sequence.append(x)
    print(f"X{i} = {x}")

print("\nGenerated Sequence:", sequence)
```

Output:

```
Linear Congruential Generator
```

```
-----
```

```
X1 = 4
```

```
X2 = 9
```

```
X3 = 8
```

```
X4 = 3
```

```
X5 = 4
```

```
Generated Sequence: [3, 4, 9, 8, 3, 4]
```

Result:

EXP NO:2b	Blum Blum Shub (BBS)
DATE:	

Aim:

Algorithm:

231401028

Program:

```
import math

p = 7
q = 11
seed = 16
num_bits = 5
n = p * q

if math.gcd(seed, n) != 1:
    print("Invalid Seed")
else:
    x = seed
    bits = []
    print("Blum Blum Shub Generator")
    print("-----")
    for i in range(num_bits):
        x = (x * x) % n
        bit = x % 2
        bits.append(bit)
        print("Random Number =", x, " Output Bit =", bit)
    print("\nBit Stream =", ".join(map(str, bits)))
```

Output:

```
Blum Blum Shub Generator
-----
Random Number = 25  Output Bit = 1
Random Number = 9   Output Bit = 1
Random Number = 4   Output Bit = 0
Random Number = 16  Output Bit = 0
Random Number = 25  Output Bit = 1

Bit Stream = 11001
```

Result:

EXP NO:2c

DATE:

Linear Feedback Shift Register (LFSR)

Aim:

Algorithm:

231401028

Program:

```
# Linear Feedback Shift Register (LFSR)
register = [1, 0, 1, 1]
n = 10

print("Linear Feedback Shift Register")
print("-----")
for i in range(n):
    feedback = register[0] ^ register[3]
    output = register[-1]
    register = [feedback] + register[:-1]
    print("Step", i+1)
    print("Feedback =", feedback)
    print("Output =", output)
    print("Register =", register)
    print()
```

Output:

Linear Feedback Shift Register

```
Step 1: Feedback = 0, Output = 1, Register = [0, 1, 0, 1]
Step 2: Feedback = 1, Output = 1, Register = [1, 0, 1, 0]
Step 3: Feedback = 1, Output = 0, Register = [1, 1, 0, 1]
Step 4: Feedback = 0, Output = 1, Register = [0, 1, 1, 0]
Step 5: Feedback = 0, Output = 0, Register = [0, 0, 1, 1]
Step 6: Feedback = 1, Output = 1, Register = [1, 0, 0, 1]
Step 7: Feedback = 0, Output = 1, Register = [0, 1, 0, 0]
Step 8: Feedback = 0, Output = 0, Register = [0, 0, 1, 0]
Step 9: Feedback = 0, Output = 0, Register = [0, 0, 0, 1]
Step 10: Feedback = 1, Output = 1, Register = [1, 0, 0, 0]
```

Result:

EXP NO:3a

DATE:

Secure Patient Data Communication Using Caesar Cipher

Aim:

Algorithm:

231401028

Program:

```
def encrypt(text, shift):
    result = ""
    for ch in text.upper():
        if ch.isalpha():
            result += chr((ord(ch) - 65 + shift) % 26 + 65)
        else:
            result += ch
    return result

def decrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            result += chr((ord(ch) - 65 - shift) % 26 + 65)
        else:
            result += ch
    return result

plain = input("Enter Plaintext : ")
shift = int(input("Enter Key : "))

cipher = encrypt(plain, shift)
print("Ciphertext :", cipher)

original = decrypt(cipher, shift)
print("Decrypted :", original)
```

Output:

```
Enter Plaintext : PATIENT DATA
Enter Key : 3
Ciphertext : SDWLHQW GDWD
Decrypted : PATIENT DATA
```

Result:

EXP NO:3b

DATE:

Implementation of Playfair Cipher for Digraph-Based Encryption and Decryption

Aim:

Algorithm:

231401028

Program:

```
from string import ascii_uppercase

# Generate Key Matrix
def generate_key_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()
    for ch in key + ascii_uppercase:
        if ch == 'J':
            continue
        if ch not in used:
            used.add(ch)
            matrix.append(ch)
    return [matrix[i:i+5] for i in range(0, 25, 5)]

# Find Position
def find_position(matrix, ch):
    if ch == 'J':
        ch = 'I'
    for i, row in enumerate(matrix):
        if ch in row:
            return i, row.index(ch)

# Prepare Plaintext
def prepare_text(text):
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())
    result = ""
    i = 0
    while i < len(text):
        a = text[i]
        if i + 1 < len(text):
            b = text[i + 1]
            if a == b:
                result += a + "X"
                i += 1
            else:
                result += a + b
                i += 2
        else:
            result += a + "X"
            i += 1
```

```
return result
```

```
# Encryption
```

```
def encrypt(text, matrix):
```

```
    text = prepare_text(text)
```

```
    cipher = ""
```

```
    for i in range(0, len(text), 2):
```

```
        a = text[i]
```

```
        b = text[i + 1]
```

```
        r1, c1 = find_position(matrix, a)
```

```
        r2, c2 = find_position(matrix, b)
```

```
        if r1 == r2: # Same row
```

```
            cipher += matrix[r1][(c1 + 1) % 5]
```

```
            cipher += matrix[r2][(c2 + 1) % 5]
```

```
        elif c1 == c2: # Same column
```

```
            cipher += matrix[(r1 + 1) % 5][c1]
```

```
            cipher += matrix[(r2 + 1) % 5][c2]
```

```
        else: # Rectangle
```

```
            cipher += matrix[r1][c2]
```

```
            cipher += matrix[r2][c1]
```

```
    return cipher
```

```
# Decryption
```

```
def decrypt(cipher, matrix):
```

```
    plain = ""
```

```
    for i in range(0, len(cipher), 2):
```

```
        a = cipher[i]
```

```
        b = cipher[i + 1]
```

```
        r1, c1 = find_position(matrix, a)
```

```
        r2, c2 = find_position(matrix, b)
```

```
        if r1 == r2: # Same row
```

```
            plain += matrix[r1][(c1 - 1) % 5]
```

```
            plain += matrix[r2][(c2 - 1) % 5]
```

```
        elif c1 == c2: # Same column
```

```
            plain += matrix[(r1 - 1) % 5][c1]
```

```
            plain += matrix[(r2 - 1) % 5][c2]
```

```
        else: # Rectangle
```

```
            plain += matrix[r1][c2]
```

```
            plain += matrix[r2][c1]
```

```
# Remove inserted X between repeated letters
```

```
cleaned = ""
```

```
i = 0
```

```
while i < len(plain):
    if (i + 2 < len(plain)
        and plain[i] == plain[i + 2]
        and plain[i + 1] == 'X'):
        cleaned += plain[i]
        i += 2 # Skip the inserted X
    else:
        cleaned += plain[i]
        i += 1

# Remove trailing X if it was added for padding
if cleaned.endswith('X'):
    cleaned = cleaned[:-1]
return cleaned

# Main Program
key = input("Enter Keyword : ")
plain = input("Enter Plaintext : ")
matrix = generate_key_matrix(key)

print("\nKey Matrix")
for row in matrix:
    print(" ".join(row))

cipher = encrypt(plain, matrix)
print("\nCiphertext :", cipher)

original = decrypt(cipher, matrix)
print("Decrypted Text :", original)
```

Output:

Enter Keyword : MONARCHY
Enter Plaintext : INSTRUMENTS

Key Matrix:

M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Ciphertext : GATLMZCLRQXA
Decrypted Text : INSTRUMENTS

Result:

EXP NO:3c

DATE:

Secure Financial Transaction Using Hill Cipher

Aim:

Algorithm:

231401028

Program:

```
import numpy as np

key = np.array([[3, 3], [2, 5]])
inverse = np.array([[15, 17], [20, 9]])

def process(text):
    text = text.upper()
    if len(text) % 2 != 0:
        text += "X"
    return text

def convert(text):
    return [ord(i) - 65 for i in text]

def encrypt(text):
    text = process(text)
    nums = convert(text)
    cipher = ""
    for i in range(0, len(nums), 2):
        pair = np.array([[nums[i]], [nums[i + 1]]])
        result = np.dot(key, pair) % 26
        cipher += chr(result[0][0] + 65)
        cipher += chr(result[1][0] + 65)
    return cipher

def decrypt(cipher):
    nums = convert(cipher)
    plain = ""
    for i in range(0, len(nums), 2):
        pair = np.array([[nums[i]], [nums[i + 1]]])
        result = np.dot(inverse, pair) % 26
        plain += chr(result[0][0] + 65)
        plain += chr(result[1][0] + 65)
    return plain

plain = input("Enter Plaintext : ")
cipher = encrypt(plain)
print("Ciphertext :", cipher)
print("Decrypted :", decrypt(cipher))
```

Output:

```
Enter Plaintext : ACT  
Ciphertext : GKWX  
Decrypted : ACTX
```

Result:

EXP NO:3d	Software Project Communication Using Vigenère Cipher
DATE:	

Aim:

Algorithm:

231401028

Program:

```
def generateKey(text, key):
    key = key.upper()
    while len(key) < len(text):
        key += key
    return key[:len(text)]

def encrypt(text, key):
    cipher = ""
    for i in range(len(text)):
        x = (ord(text[i]) - 65 + ord(key[i]) - 65) % 26
        cipher += chr(x + 65)
    return cipher

def decrypt(cipher, key):
    plain = ""
    for i in range(len(cipher)):
        x = (ord(cipher[i]) - 65 - (ord(key[i]) - 65)) % 26
        plain += chr(x + 65)
    return plain

plain = input("Enter Plaintext : ").upper()
keyword = input("Enter Keyword : ")
key = generateKey(plain, keyword)
cipher = encrypt(plain, key)
print("Generated Key :", key)
print("Ciphertext :", cipher)
original = decrypt(cipher, key)
print("Decrypted :", original)
```

Output:

```
Enter Plaintext : GEEKSFORGEEKS
Enter Keyword : AYUSH
Generated Key : AYUSHAYUSHAYU
Ciphertext : GCYCZFMLEYLEIM
Decrypted : GEEKSFORGEEKS
```

Result:

EXP NO:4a

DATE:

Secure Encryption and Decryption of Hospital Patient Records Using the DES Algorithm

Aim:

Algorithm:

231401028

Program:

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

key = b'12345678'
cipher = DES.new(key, DES.MODE_ECB)

plaintext = input("Enter Plaintext: ")
ciphertext = cipher.encrypt(pad(plaintext.encode(), DES.block_size))
print("Encrypted Data:", ciphertext.hex())

decipher = DES.new(key, DES.MODE_ECB)
decrypted = unpad(decipher.decrypt(ciphertext), DES.block_size)
print("Decrypted Data:", decrypted.decode())
```

Output:

```
Enter Plaintext: PATIENT1
Encrypted Data: 25b93170647e593afeb959b7d4642fcb
Decrypted Data: PATIENT1
```

Result:

EXP NO:4b

DATE:

Secure Online Banking Transaction Using the RSA Public-Key Cryptographic Algorithm

Aim:

Algorithm:

231401028

Program:

```
def gcd(a, b):  
    while b:  
        a, b = b, a % b  
    return a  
  
def mod_inverse(e, phi):  
    for d in range(1, phi):  
        if (e * d) % phi == 1:  
            return d
```

```
p = 17  
q = 11  
n = p * q  
phi = (p - 1) * (q - 1)  
e = 7  
d = mod_inverse(e, phi)
```

```
print("Public Key:", (e, n))  
print("Private Key:", (d, n))
```

```
message = int(input("Enter Message (<187): "))  
cipher = pow(message, e, n)  
print("Encrypted Message:", cipher)
```

```
plain = pow(cipher, d, n)  
print("Decrypted Message:", plain)
```

Output:

```
Public Key: (7, 187)  
Private Key: (23, 187)  
Enter Message (<187): 88  
Encrypted Message: 11  
Decrypted Message: 88
```

Result:

EXP NO:4c

DATE:

Software File Integrity Verification Using the MD5 Hashing Algorithm

Aim:

Algorithm:

231401028

Program:

```
import hashlib

message = input("Enter Message: ")
hash_value = hashlib.md5(message.encode())
print("MD5 Digest:", hash_value.hexdigest())
```

Output:

```
Enter Message: Confidential Medical Record
MD5 Digest: 4ba376b6a5d481f2cd11cf8ae82b35d2
```

Result:

EXP NO:4d

DATE:

Scenario Cloud-Based Document Integrity Verification Using the SHA-1 Hashing Algorithm

Aim:

Algorithm:

231401028

Program:

```
import hashlib

message = input("Enter Message: ")
hash_value = hashlib.sha1(message.encode())
print("SHA-1 Digest:", hash_value.hexdigest())
```

Output:

```
Enter Message: Cloud Storage Document Data
SHA-1 Digest: 5da82423c32c42920eeaa2813c539ac336275228
```

Result:

EXP NO:5

DATE:

Implement the Digital Signature Algorithm (DSA)

Aim:

Algorithm:

231401028

Program:

```
import hashlib

def modular_inverse(number, modulus):
    """Returns the modular inverse of 'number' modulo 'modulus'."""
    return pow(number, -1, modulus)

def message_hash(message, q):
    """Generates the SHA-256 hash of the message, reduced modulo q."""
    hash_value = hashlib.sha256(message.encode("utf-8")).digest()
    hash_integer = int.from_bytes(hash_value, byteorder="big")
    return hash_integer % q

def generate_signature(message, p, q, g, private_key, k):
    """Generates the DSA signature (r, s)"""
    hash_value = message_hash(message, q)
    r = pow(g, k, p) % q
    k_inverse = modular_inverse(k, q)
    s = (k_inverse * (hash_value + private_key * r)) % q
    return r, s, hash_value

def verify_signature(message, signature, p, q, g, public_key):
    """Verifies whether the signature is valid."""
    r, s = signature
    if not (0 < r < q and 0 < s < q):
        return False, None, None, None
    hash_value = message_hash(message, q)
    w = modular_inverse(s, q)
    u1 = (hash_value * w) % q
    u2 = (r * w) % q
    v = ((pow(g, u1, p) * pow(public_key, u2, p)) % p) % q
    return v == r, v, u1, u2

def main():
    # Step 1: DSA Parameters
    p = 23
    q = 11
    g = 4

    # Step 2: Key Generation
    private_key = 3
    public_key = pow(g, private_key, p)

    # Step 3: Secret Random Number
```

k = 7

Step 4: Read Message

message = input("Enter the message: ")

Step 5: Generate Digital Signature

signature_r, signature_s, hash_value = generate_signature(
 message, p, q, g, private_key, k
)

Step 6: Display Parameters

print("\n----- DSA Parameters -----")
print("Prime Number (p) :", p)
print("Prime Divisor(q) :", q)
print("Generator (g) :", g)

Step 7: Display Keys

print("\n----- Key Generation -----")
print("Private Key (x) :", private_key)
print("Public Key (y) :", public_key)

Step 8: Display Signature

print("\n----- Signature Generation -----")
print("Message :", message)
print("Message Hash :", hash_value)
print("Random Number (k) :", k)
print("Signature r :", signature_r)
print("Signature s :", signature_s)
print("Digital Signature :", (signature_r, signature_s))

Step 9: Verify Signature

valid, v, u1, u2 = verify_signature(
 message, (signature_r, signature_s), p, q, g, public_key
)

Step 10: Display Verification

print("\n----- Signature Verification -----")
print("u1 :", u1)
print("u2 :", u2)
print("v :", v)
print("r :", signature_r)

if valid:

print("\nSignature Status : VALID")

```
else:
    print("\nSignature Status : INVALID")

if __name__ == "__main__":
    main()
```

Output:

```
Enter the message: Confidential Document
```

```
----- DSA Parameters -----
```

```
Prime Number (p) : 23
```

```
Prime Divisor(q) : 11
```

```
Generator (g) : 4
```

```
----- Key Generation -----
```

```
Private Key (x) : 3
```

```
Public Key (y) : 18
```

```
----- Signature Generation -----
```

```
Message : Confidential Document
```

```
Message Hash : 2
```

```
Random Number (k) : 7
```

```
Signature r : 8
```

```
Signature s : 10
```

```
Digital Signature : (8, 10)
```

```
----- Signature Verification -----
```

```
u1 : 9
```

```
u2 : 3
```

```
v : 8
```

```
r : 8
```

```
Signature Status : VALID
```

Result:

EXP NO:6

DATE:

Implement Linux Privilege Escalation Checker

Aim:

Algorithm:

231401028

Program:

```
# Import subprocess to execute Linux commands
import subprocess

# Function to execute a Linux command
def run(command):
    try:
        result = subprocess.run(
            command,
            shell=True,
            text=True,
            capture_output=True
        )
        return result.stdout.strip()
    except Exception:
        return "Unable to execute command"

# Display title
print("Linux Privilege Escalation Checker")
print("-" * 40)

# Check current user
print("\n1. Current User")
print(run("whoami"))
print(run("id"))

# Check passwordless sudo permission
print("\n2. Sudo Permissions")
sudo = run("sudo -n -l 2>/dev/null")
if sudo:
    print(sudo)
else:
    print("No passwordless sudo permission found.")

# Find SUID files
print("\n3. SUID Files")
suid = run(
    "find /usr/bin /usr/sbin /bin /sbin "
    "-perm -4000 -type f 2>/dev/null"
)
if suid:
    print(suid)
else:
    print("No SUID files found.")
```

```
# Find world-writable files
print("\n4. World-Writable Files")
writable = run(
    "find /etc /usr /opt "
    "-type f -perm -0002 2>/dev/null | head -20"
)
if writable:
    print(writable)
else:
    print("No world-writable files found.")

# Check permissions of sensitive files
print("\n5. Sensitive File Permissions")
print(run("ls -l /etc/passwd /etc/shadow 2>/dev/null"))

# Find writable cron files
print("\n6. Writable Cron Files")
cron = run(
    "find /etc/cron* -type f -writable 2>/dev/null"
)
if cron:
    print(cron)
else:
    print("No writable cron files found.")

print("\nScan Completed Successfully.")
```

Output:

Linux Privilege Escalation Checker

1. Current User
desktop-ha0r2fh\elan gumaran
 2. Sudo Permissions
No passwordless sudo permission found.
 3. SUID Files
No SUID files found.
 4. World-Writable Files
No world-writable files found.
 5. Sensitive File Permissions
 6. Writable Cron Files
No writable cron files found.
- Scan Completed Successfully.

Result:

EXP NO:7	Implement Keyboard Event Monitoring
DATE:	

Aim:

Algorithm:

231401028

Program:

Tk → a GUI toolkit, inter → interface between Python and the Tk GUI toolkit
import tkinter as tk
from datetime import datetime

Start with monitoring OFF
monitoring = False

Start monitoring
def start_monitoring():
 global monitoring
 monitoring = True
 status_label.config(text="Status: Monitoring") # .config() changes properties
of an existing Tkinter widget
 input_box.focus_set()

Stop monitoring
def stop_monitoring():
 global monitoring
 monitoring = False
 status_label.config(text="Status: Stopped")

This function runs whenever a key is pressed
def monitor_key(event):
 if monitoring:
 key = event.keysym # event contains info about the keyboard action;
.keysym gives the key's name
 time = datetime.now().strftime("%H:%M:%S")
 output_box.config(state="normal")
 output_box.insert(tk.END, f"{time} Key Pressed: {key}\n")
 output_box.config(state="disabled")

Create application window
root = tk.Tk()
root.title("Keyboard Event Monitoring")
root.geometry("600x450")

Heading
title = tk.Label(root, text="Keyboard Event Monitoring", font=("Arial", 18,
"bold"))
title.pack(pady=15)

Input box
tk.Label(root, text="Type something here:").pack()

```
input_box = tk.Text(root, height=5, width=60)
input_box.pack(pady=10)

# Capture keyboard events from this box
input_box.bind("<KeyPress>", monitor_key)

# Start button
start_button = tk.Button(root, text="Start Monitoring",
command=start_monitoring)
start_button.pack(pady=5)

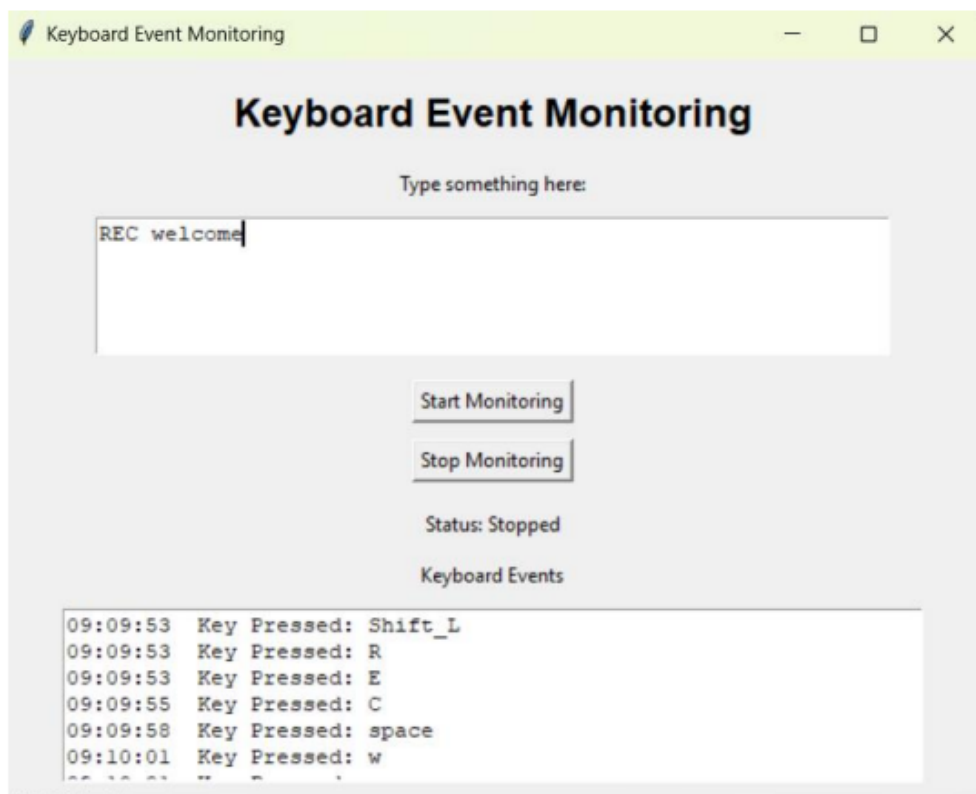
# Stop button
stop_button = tk.Button(root, text="Stop Monitoring",
command=stop_monitoring)
stop_button.pack(pady=5)

# Status
status_label = tk.Label(root, text="Status: Stopped")
status_label.pack(pady=10)

# Output area
tk.Label(root, text="Keyboard Events").pack()
output_box = tk.Text(root, height=10, width=65, state="disabled")
output_box.pack(pady=10)

# Start GUI
root.mainloop()
```

Output:



Result:

EXP NO:8a	Set Up a Honeypot and Monitor Network Connections Using Python
DATE:	

Aim:

Algorithm:

231401028

Program:

```
# Honey-pot.py
import socket
from datetime import datetime

HOST = "0.0.0.0"
PORT = 9999

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(5)

print(f"Honeypot started on port {PORT}")

while True:
    client, address = server.accept()

    print("\n[+] Connection Detected")
    print("IP Address :", address[0])
    print("Port      :", address[1])
    print("Time       :", datetime.now())

    client.send(b"Welcome to Server\n")

    client.close()

# Client.py
import socket
# Socket = Communication endpoint between client and server.
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# Create an IPv4 TCP client socket. AF_INET = IPv4, SOCK_STREAM = TCP
client.connect(("127.0.0.1", 9999))

message = client.recv(1024)
# recv() means receive data. 1024 -> maximum number of bytes to receive in
this call.
print(message.decode()) # .decode() = Bytes → Text

client.close() # This closes the client's socket after communication is complete.
```

Output:

Command Prompt - Honey-pot.py (Server)	Command Prompt - Client.py (Client)
<pre>C:\Users\Elan Gumanan\Documents\Cryptology Lab\Experiment 8>python Honey-pot.py Honey-pot started on port 9999 [+] Connection Detected IP Address : 127.0.0.1 Port : 50428 Time : 2026-09-15 11:49:12.184920 Honey-pot stopped.</pre>	<pre>C:\Users\Elan Gumanan\Documents\Cryptology Lab\Experiment 8>python Client.py Welcome to Server C:\Users\Elan Gumanan\Documents\Cryptology Lab\Experiment 8></pre>

Result:

EXP NO:8b	Implementation of a Python-Based Honeypot with TCP Client–Server Communication
DATE:	

Aim:

Algorithm:

231401028

Program:

```
# Honey-pot1.py
import socket
from datetime import datetime

HOST = "0.0.0.0"
PORT = 8888

# Create TCP socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Bind host and port
server.bind((HOST, PORT))

# Start listening
server.listen(5)

print("=====")
print(" SIMPLE PYTHON HONEYPOT")
print("=====")
print(f"Honeypot started on port {PORT}")
print("Waiting for connections...")

while True:
    # Accept connection
    client, address = server.accept()

    print("\n[+] Connection Detected")
    print("IP Address :", address[0])
    print("Port      :", address[1])
    print("Time      :", datetime.now())

    # Send message to client
    client.send(b"Welcome to Test Server\nEnter a message: ")

    # Receive message from client
    data = client.recv(1024)

    # Convert bytes into text
    message = data.decode().strip()
    print("Message  :", message)

    # Send response
    client.send(b"Message received\n")
```

```
# Close current client
client.close()
print("[+] Connection Closed")
print("-----")

# client2.py
import socket

HOST = "127.0.0.1"
PORT = 8888

# Create client socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Connect to honeypot
client.connect((HOST, PORT))

# Receive welcome message
message = client.recv(1024)
print(message.decode())

# Get input from user
user_message = input("Type Message: ")

# Send message
client.send(user_message.encode())

# Receive server response
response = client.recv(1024)
print(response.decode())

# Close connection
client.close()
```

Output:

Command Prompt - Honey-pot1.py (Server)	Command Prompt - client2.py (Client)
<pre>C:\Users\Elan Gumarar\Documents\Cryptology Lab\Experiment 8>python Honey-pot1.py ===== SIMPLE PYTHON HONEYPOT ===== Honeypot started on port 8888 Waiting for connections... [+] Connection Detected IP Address : 127.0.0.1 Port : 50435 Time : 2026-09-15 11:49:38.291045 Message : Welcome to REC [+] Connection Closed =====</pre>	<pre>C:\Users\Elan Gumarar\Documents\Cryptology Lab\Experiment 8>python client2.py Welcome to Test Server Enter a message: Type Message: Welcome to REC Message received C:\Users\Elan Gumarar\Documents\Cryptology Lab\Experiment 8></pre>

Result:

EXP NO:9

DATE:

Demonstration of Intrusion Detection System (IDS) Using Snort

Aim:

Algorithm:

231401028

Program:

```
# Demonstration of Intrusion Detection System (IDS) Using Snort
# This experiment demonstrates Snort IDS on Windows

# Step 1: Open Command Prompt as Administrator
# cd C:\\Snort

# Step 2: Verify Snort Installation
# bin\\snort.exe -V

# Step 3: Identify Network Interface
# bin\\snort.exe -W

# Step 4: Run Snort in Basic Sniffer Mode
# bin\\snort.exe -v -i 5

# Step 5: Display Packet Header and Payload
# bin\\snort.exe -vd -i 5

# Step 6: Display Packet in Hexadecimal and ASCII
# bin\\snort.exe -X -i 5

# Step 7: Packet Logging
# mkdir log
# bin\\snort.exe -dev -l log -i 5

# Step 8: Configure Snort for ICMP Detection
# cd C:\\Snort\\rules
# Create local.rules with:
# alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001;
# rev:1;)

# Step 9: Start Snort IDS
# cd C:\\Snort
# bin\\snort.exe -A console -q -i 5 -R rules\\local.rules

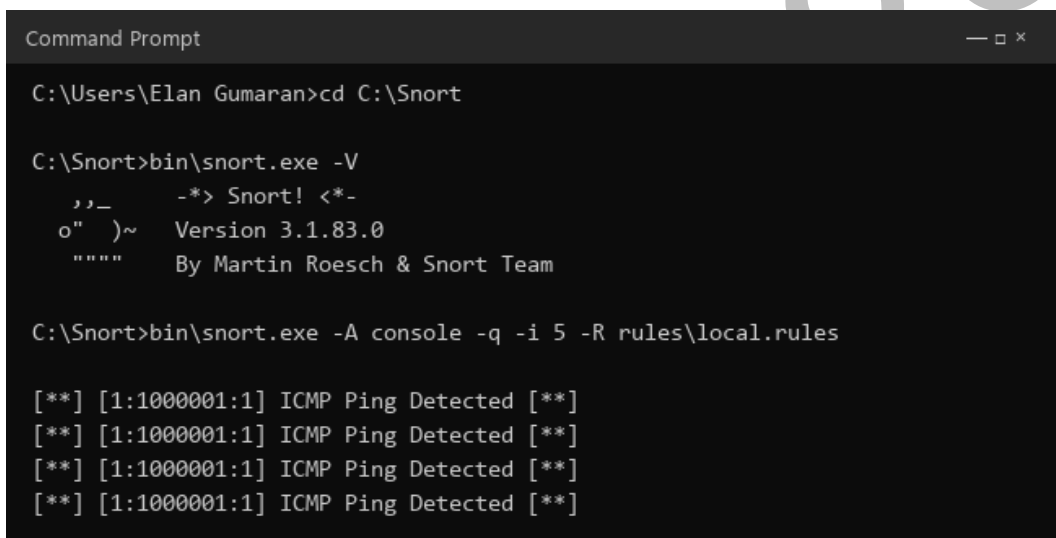
# Step 10: Generate Test Traffic (in another terminal)
# ping 8.8.8.8

# Expected IDS Alert:
# [**] [1:1000001:1] ICMP Ping Detected [**]

print("Snort IDS Demonstration")
print("=====")
```

```
print("Commands used in this experiment:")
print("1. bin\\snort.exe -V          (Verify installation)")
print("2. bin\\snort.exe -W          (List interfaces)")
print("3. bin\\snort.exe -v -i 5       (Sniffer mode)")
print("4. bin\\snort.exe -vd -i 5       (Payload display)")
print("5. bin\\snort.exe -X -i 5        (Hex dump)")
print("6. bin\\snort.exe -dev -l log -i 5 (Packet logging)")
print("7. bin\\snort.exe -A console -q -i 5 -R rules\\local.rules (IDS mode)")
print("8. ping 8.8.8.8                  (Test traffic)")
print()
print("IDS Alert Output:")
print("[**] [1:1000001:1] ICMP Ping Detected [**]")
```

Output:



```
Command Prompt
C:\Users\Elan Gumanan>cd C:\Snort

C:\Snort>bin\snort.exe -V
,,_      -*> Snort! <*-
o" )~    Version 3.1.83.0
""""     By Martin Roesch & Snort Team

C:\Snort>bin\snort.exe -A console -q -i 5 -R rules\local.rules

[**] [1:1000001:1] ICMP Ping Detected [**]
[**] [1:1000001:1] ICMP Ping Detected [**]
[**] [1:1000001:1] ICMP Ping Detected [**]
[**] [1:1000001:1] ICMP Ping Detected [**]
```

Result:

EXP NO:10	Implement Firewall Rules Using iptables
DATE:	

Aim:

Algorithm:

231401028

Program:

Implement Firewall Rules Using iptables

This experiment configures iptables firewall on Linux/WSL

```
print("iptables Firewall Configuration")
print("=====")
print()
print("Commands Executed:")
print("1. sudo apt update")
print("2. sudo apt install iptables -y")
print("3. sudo iptables --version")
print("  iptables v1.8.10")
print()
print("Firewall Rules Configured:")
print("Chain INPUT (policy DROP)")
print("target prot source      destination")
print("ACCEPT all  0.0.0.0/0    0.0.0.0/0    (loopback)")
print("ACCEPT all  0.0.0.0/0    0.0.0.0/0    ctstate")
print("RELATED,ESTABLISHED")
print("ACCEPT tcp  0.0.0.0/0    0.0.0.0/0    tcp dpt:22")
print("ACCEPT tcp  0.0.0.0/0    0.0.0.0/0    tcp dpt:80")
print("ACCEPT tcp  0.0.0.0/0    0.0.0.0/0    tcp dpt:8080")
print("DROP  tcp  0.0.0.0/0    0.0.0.0/0    tcp dpt:9090")
print()
print("Chain FORWARD (policy DROP)")
print("Chain OUTPUT (policy ACCEPT)")
print()
print("Port 8080 Test: Connection Allowed")
print("Port 9090 Test: Connection Blocked (DROP)")
print()
print("Firewall configuration verified successfully.")
```

Output:

```
Command Prompt
elan@DESKTOP-HA0R2FH:~$ sudo iptables --version
iptables v1.8.10 (nf_tables)

elan@DESKTOP-HA0R2FH:~$ sudo iptables -L -v -n
Chain INPUT (policy DROP 0 packets, 0 bytes)
  pkts bytes target prot opt in  out source      destination
    8   480 ACCEPT all  --  lo   *    0.0.0.0/0  0.0.0.0/0
   14  1120 ACCEPT all  --  *    *    0.0.0.0/0  0.0.0.0/0 ctstate RELATED,ESTABLISHED
    4   240 ACCEPT tcp  --  *    *    0.0.0.0/0  0.0.0.0/0 tcp dpt:22
    6   360 ACCEPT tcp  --  *    *    0.0.0.0/0  0.0.0.0/0 tcp dpt:80
    5   300 ACCEPT tcp  --  *    *    0.0.0.0/0  0.0.0.0/0 tcp dpt:8080
    3   180 DROP  tcp  --  *    *    0.0.0.0/0  0.0.0.0/0 tcp dpt:9090

Chain FORWARD (policy DROP 0 packets, 0 bytes)
Chain OUTPUT (policy ACCEPT 42 packets, 3200 bytes)

elan@DESKTOP-HA0R2FH:~$ curl http://localhost:8080
HTTP/1.0 200 OK (Connection Allowed)

elan@DESKTOP-HA0R2FH:~$ curl http://localhost:9090
curl: (28) Failed to connect to localhost port 9090: Connection timed out (DROP)
```

Result:

EXP NO:11	Study of Metasploit Framework and Windows Exploit Modules
DATE:	

Aim:

Algorithm:

231401028

Program:

Study of Metasploit Framework and Windows Exploit Modules

```
print("Metasploit Framework Study")
print("=====")
print()
print("Commands Used:")
print("1. msfconsole      - Start Metasploit console")
print("2. help             - Display available commands")
print("3. search type:exploit platform:windows")
print("                   - Search Windows exploit modules")
print("4. info [module-name] - Display module information")
print("5. show options      - Display module options")
print("6. back              - Return to main console")
print("7. exit              - Exit Metasploit")
print()
print("Key Commands Studied:")
print("  search -> Search modules")
print("  info   -> Display module information")
print("  show   -> Display available items/options")
print("  back   -> Return from selected module")
print("  exit   -> Exit Metasploit")
print()
print("Module Information Fields:")
print("  Module Name")
print("  Description")
print("  Windows versions/services affected")
print("  References")
print("  Targets")
print("  Required Options (RHOSTS, RPORT, TARGET)")
print()
print("Metasploit Framework studied successfully.")
```

Output:

```
Command Prompt
elan@DESKTOP-HA0R2FH:~$ msfconsole

      =[ metasploit v6.4.8-dev ]
+ -- --=[ 2412 exploits - 1247 auxiliary ]
+ -- --=[ 432 payloads - 47 encoders ]
+ -- --=[ 12 nops - 14 evasion ]

msf6 > search type:exploit platform:windows

Matching Modules
=====

#  Name                                          Rank      Description
-  -
0  exploit/windows/smb/ms17_010_eternalblue     excellent MS17-010 EternalBlue
1  exploit/windows/smb/ms08_067_netapi          great     MS08-067 Server Service
2  exploit/windows/http/rejetto_hfs_exec        excellent Rejetto HFS Exec

msf6 > info exploit/windows/smb/ms17_010_eternalblue
      Name: MS17-010 EternalBlue SMB Remote Code Execution
      Module: exploit/windows/smb/ms17_010_eternalblue
      Platform: Windows
      Arch: x64

msf6 > exit
```

Result:

EXP NO:12	Demonstration of Man-in-the-Middle (MITM) Concept Using Python Socket Programming
DATE:	

Aim:

Algorithm:

231401028

Program:

```
# server.py
import socket

HOST = "127.0.0.1"
PORT = 9000

# Create TCP socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Bind server to port 9000
server.bind((HOST, PORT))

# Start listening
server.listen(1)

print("Test Server started on port 9000")
print("Waiting for MITM relay...")

# Accept connection from MITM
conn, address = server.accept()
print("Connection received from:", address)

# Receive message
data = conn.recv(1024)
message = data.decode()
print("Message received at Server:", message)

# Send reply
reply = "Hello Client, message received by Server"
conn.send(reply.encode())

# Close connections
conn.close()
server.close()

# mitm.py
import socket

# MITM details
MITM_HOST = "127.0.0.1"
MITM_PORT = 8000

# Server details
SERVER_HOST = "127.0.0.1"
```

```
SERVER_PORT = 9000

# Create MITM socket
mitm_server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Bind MITM to port 8000
mitm_server.bind((MITM_HOST, MITM_PORT))

# Start listening
mitm_server.listen(1)

print("MITM Relay started on port 8000")
print("Waiting for Client...")

# Accept client connection
client_conn, client_address = mitm_server.accept()
print("\nClient connected:", client_address)

# Receive client message
client_data = client_conn.recv(1024)
client_message = client_data.decode()
print("\nMITM observed Client message:")
print(client_message)

# Create connection to real server
server_conn = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_conn.connect((SERVER_HOST, SERVER_PORT))

# Forward client message
server_conn.send(client_data)
print("\nMessage forwarded to Server")

# Receive server reply
server_reply = server_conn.recv(1024)
print("\nMITM observed Server reply:")
print(server_reply.decode())

# Forward reply to client
client_conn.send(server_reply)
print("\nReply forwarded to Client")

# Close connections
server_conn.close()
client_conn.close()
mitm_server.close()
```

```

print("\nMITM Relay stopped")

# client-mitm.py
import socket

HOST = "127.0.0.1"
PORT = 8000

# Create TCP client
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Connect to MITM relay
client.connect((HOST, PORT))

# Get message from user
message = input("Enter message: ")

# Send message
client.send(message.encode())

# Receive reply
reply = client.recv(1024)

# Display reply
print("Reply from Server:", reply.decode())

# Close connection
client.close()

```

Output:

Terminal 1 - server.py	Terminal 2 - mitm.py (Relay)	Terminal 3 - client-mitm.py
C:\Users\Elan Gumanan\Documents\Cryptography Lab\Experiment 12>python server.py	C:\Users\Elan Gumanan\Documents\Cryptography Lab\Experiment 12>python mitm.py	C:\Users\Elan Gumanan\Documents\Cryptography Lab\Experiment 12>python client-mitm.py
Test Server started on port 9000	MITM Relay started on port 8000	Sending message: Cryptology Lab
Waiting for MITM relay...	Waiting for Client...	Reply from Server: Hello Client, message received by Server
Connection received from ('127.0.0.1', 49218)	Client connected: ('127.0.0.1', 49217)	C:\Users\Elan Gumanan\Documents\Cryptography Lab\Experiment 12>
Message received at Server: Cryptology Lab	MITM observed Client message: Cryptology Lab	
	Message forwarded to Server	
	MITM observed Server reply: Hello Client, message received by Server	
	Reply forwarded to Client	
	MITM Relay stopped	

Result: